Intelligent Research Topic Analysis & Agentic AI Research Assistant

Milestone 2: Agentic AI Research Assistant

End-Semester Project Report

Generative AI Course

Krishiv Saumya Kumar Harshvardhan Gupta

github.com/oksaumya/ARIA

April 2026

Abstract

This report presents the design and implementation of **ARIA** (Autonomous Research Intelligence Assistant), the second and final milestone of a two-part Generative AI course project. Building upon the classical NLP baseline established in Milestone 1 — which employed TF-IDF keyword extraction, LDA topic modelling, and TextRank extractive summarisation on user-supplied documents — ARIA extends the system into a fully autonomous agentic research assistant capable of answering open-ended research queries from scratch.

The agent accepts any natural language research query, autonomously decomposes it into 3–5 targeted sub-queries using *Llama 3.3 70B* (served free-of-charge through the Groq API), retrieves live information from the web via DuckDuckGo and Wikipedia, fetches and parses full page content, summarises each source in context, and iteratively validates research coverage before generating a structured Markdown report. The agentic workflow is orchestrated using **LangGraph**, with explicit state management via a typed `ResearchState` TypedDict. A grounded follow-up Q&A chat interface and one-click PDF export are provided as extensions. The application is deployed publicly on Streamlit Community Cloud, satisfying all mandatory hosting requirements. This milestone represents the culmination of the project, demonstrating the full progression from classical NLP pipelines to autonomous, multi-step agentic AI workflows.

Contents

1 Introduction **2**

1.1 Scope and Constraints 2

2 Related Work **3**

2.1 Retrieval-Augmented Generation (RAG) 3

2.2 Agentic AI Workflows . . . 3
2.3 Automated Research Assistants . . . 3

3 System Architecture **3**
3.1 High-Level Architecture . . . 3
3.2 State Design . . . 4
3.3 Project Structure . . . 5

4 Methodology **6**
4.1 Node 1 — Research Planning (`plan_research`) . . . 6
4.2 Node 2 — Web Search (`web_searcher`) . . . 6
4.3 Node 3 — Content Fetching (`content_fetcher`) . . . 6
4.4 Node 4 — Source Summarisation (`source_summarizer`) . . . 6
4.5 Node 5 — Aggregation (`aggregator`) . . . 7
4.6 Node 6 — Quality Validation (`quality_validator`) . . . 7
4.7 Node 7 — Report Generation (`report_generator`) . . . 7

5 Extension — Follow-up Q&A **7**

6 Implementation Details **8**
6.1 Technology Stack . . . 8
6.2 LLM Integration . . . 8
6.3 Robustness and Error Handling . . . 9
6.4 Source Quality Scoring . . . 9
6.5 Deployment . . . 9

7 Input–Output Specification **10**

8 Results and Evaluation **10**
8.1 Functional Output Description . . . 10
8.2 Sample Test Cases . . . 11
8.3 Limitations . . . 11

9 Comparison: Milestone 1 vs Milestone 2 **12**

10 Conclusion **13**

1 Introduction

The exponential growth of research literature across all domains demands tools that can autonomously locate, retrieve, synthesise, and present information at scale. Milestone 1 of this project established a classical NLP baseline — a pipeline comprising TF-IDF keyword extraction, LDA topic modelling, and TextRank extractive summarisation — that operates entirely on user-supplied or Wikipedia-sourced documents. While effective for local document analysis, this approach cannot answer open-ended research questions, cannot access live information, and cannot reason across multiple heterogeneous sources.

Milestone 2 addresses these limitations by transforming the system into a fully autonomous **agentic AI research assistant**. The agent, named **ARIA** (Autonomous Research Intelligence Assistant), can:

- Accept any open-ended research query in natural language.
- Decompose the query into 3–5 targeted sub-queries using an LLM planner.
- Search the live web via DuckDuckGo and Wikipedia.
- Fetch and parse full page content from retrieved URLs.
- Summarise each source in context using *Llama 3.3 70B* (Groq free tier).
- Aggregate per-source summaries into a unified synthesis.
- Validate coverage quality and iteratively retry if coverage is insufficient.
- Generate a structured research report (Title, Abstract, Key Findings, Conclusion, Sources).
- Answer follow-up questions grounded in the report.
- Export the full report and Q&A history as a PDF.

1.1 Scope and Constraints

The system was built under the following course-imposed constraints:

- **LangGraph** is mandatory as the agentic orchestration framework.
- All LLMs must be free-tier or open-source only. ARIA uses the Groq free tier (Llama 3.3 70B Versatile).
- Web search must use free APIs; ARIA uses DuckDuckGo (`ddgs`) and the `wikipedia` Python package (no API key required for either).
- The application must be hosted on a public platform (Streamlit Community Cloud). Localhost-only demonstrations are not accepted.
- Team size: 3 students.

2 Related Work

2.1 Retrieval-Augmented Generation (RAG)

ARIA's architecture is conceptually related to Retrieval-Augmented Generation (RAG), in which an LLM conditions its generation on retrieved documents rather than purely on parametric knowledge. Unlike vanilla RAG, ARIA augments retrieval with LLM-powered query planning and iterative quality validation, resulting in a multi-step agentic loop rather than a single retrieval–generation pass.

2.2 Agentic AI Workflows

ReAct introduced the paradigm of interleaving reasoning traces with tool-use actions, demonstrating that LLMs can effectively orchestrate external tools. ARIA operationalises this insight through LangGraph nodes that each perform a discrete reasoning or action step. LangGraph provides stateful graph orchestration with explicit edge conditions, enabling the iterative retry loop that distinguishes ARIA from simpler single-pass pipelines.

2.3 Automated Research Assistants

Recent systems such as GPT Researcher demonstrate fully autonomous research pipelines over the web. ARIA is constrained to free-tier APIs and open-source tooling, making it a lightweight, deployable alternative that preserves the core agentic loop within practical resource limits.

3 System Architecture

3.1 High-Level Architecture

ARIA follows a directed, stateful graph architecture orchestrated by LangGraph. The research pipeline is modelled as a `StateGraph` where each node is a pure function that reads from and writes to a shared `ResearchState` TypedDict. The graph topology is shown below.

```
plan_research
     |
     v
web_searcher <---------------------+
     |                             |
     v                             |
content_fetcher              increment_retry
     |                             ^
     v                             |
source_summarizer           [score < 0.6 AND
     |                        retry < max]
     v                             |
 aggregator                        |
     |                             |
     v                             |
quality_validator ------------------+
     |
     | [score >= 0.6 OR retry == max]
     v
report_generator
     |
     v
    END
```

Figure 1: LangGraph workflow for ARIA. Conditional edges implement iterative quality control.

3.2 State Design

All inter-node communication passes through a single `ResearchState` TypedDict (defined in `src/agent/state.py`). Key fields are listed in Table 1.

Table 1: Key fields of `ResearchState`

Field	Type	Description
`query`	`str`	Original user research query
`search_queries`	`List[str]`	LLM-generated sub-queries
`search_results`	`List[SearchResult]`	Deduplicated web results (max 15)
`failed_urls`	`List[str]`	URLs that could not be fetched
`source_summaries`	`List[str]`	Per-source LLM summaries
`aggregated_text`	`str`	Unified synthesis paragraph
`quality_score`	`float`	Coverage score in $[0, 1]$
`missing_aspects`	`List[str]`	Gaps identified by validator
`retry_count`	`int`	Number of retry iterations
`max_retries`	`int`	User-configurable ceiling (default 2)
`report`	`ResearchReport`	Structured final report
`report_markdown`	`str`	Rendered Markdown string
`qa_history`	`List[dict]`	Follow-up Q&A pairs
`errors`	`Annotated[List[str], operator.add]`	Accumulated error log
`api_key`	`str`	Runtime Groq API key

3.3 Project Structure

```
ARIA/
  app.py                        # Streamlit multi-page entry point
  setup.sh                      # One-shot environment bootstrap
  requirements.txt
  pages/
    1_NLP_Analysis.py           # Milestone 1 UI (unchanged)
    2_AI_Research_Agent.py      # Milestone 2 ARIA agent UI
  src/
    data_fetcher.py             # PDF / DOCX / TXT / Wikipedia
    nlp_pipeline.py             # TF-IDF, LDA, TextRank (M1)
    agent/
      state.py                  # ResearchState TypedDict
      graph.py                  # Graph topology & conditional edges
      nodes.py                  # 7 node functions
      llm.py                    # Groq ChatOpenAI client
```

```
    web_search.py              # DuckDuckGo & Wikipedia wrappers
    report.py                  # Markdown renderer + fpdf2 PDF export
    memory.py                  # Session history (st.session_state)
    qa_agent.py                # Follow-up Q&A grounded in report
```

Listing 1: Repository layout for ARIA

4 Methodology

4.1 Node 1 — Research Planning (`plan_research`)

The planning node receives the raw user query and instructs Llama 3.3 70B to decompose it into 3–5 targeted sub-queries that cover distinct facets of the topic: general overview, recent developments, applications, criticisms or limitations, and future directions. The LLM is prompted to return a JSON object with a single `queries` key. A robust `_extract_json_object` helper strips Markdown code fences and extracts the innermost JSON object from the response to tolerate common LLM formatting quirks. If planning fails, the original query is used as a fallback single sub-query and the error is logged to `state["errors"]`.

4.2 Node 2 — Web Search (`web_searcher`)

The search node executes each sub-query against two sources in parallel:

DuckDuckGo (`DuckDuckGoSearcher`)
: Uses the `ddgs` library with up to 5 results per query and a two-backend retry strategy (`html` → `api`). Results are deduplicated by URL and capped at 15 total.

Wikipedia (`WikipediaSearcher`)
: Uses the `wikipedia` Python package to fetch up to 3 article summaries per query. Wikipedia results are treated as high-trust sources.

Each `SearchResult` records a heuristic `source_quality` score in $[0, 1]$ computed from the URL's scheme (HTTPS bonus), domain trust tier (academic, government, Wikipedia), and path depth. This score is available for downstream processing.

On retry iterations, `missing_aspects` identified by the quality validator are appended to the query list so that the agent targets its gaps specifically.

4.3 Node 3 — Content Fetching (`content_fetcher`)

For each search result that does not yet have `full_content`, the node issues an HTTP GET request using `urllib3` (3 s connect / 5 s read timeout). The response HTML is parsed with BeautifulSoup: `<script>`, `<style>`, `<nav>`, `<footer>`, and `<header>` tags are stripped, and all `<p>` paragraph texts are joined and truncated to 3,000 characters. Failed URLs are recorded in `state["failed_urls"]` so that retry iterations do not re-attempt them; such results fall back to their search snippet.

4.4 Node 4 — Source Summarisation (`source_summarizer`)

Each fetched source is summarised in 3–5 sentences by Llama 3.3 70B, prompted to focus specifically on the relevance of the content to the original query. Sources that are irrelevant

to the query are discarded (the LLM is instructed to reply `IRRELEVANT` for such cases). A 1-second inter-request sleep respects Groq's free-tier rate limits. If summarisation fails for a source, the raw snippet is used as a fallback.

4.5 Node 5 — Aggregation (`aggregator`)

All per-source summaries are concatenated and provided to the LLM with a synthesis prompt. The model is asked to write 3–5 paragraphs that: (a) identify consensus findings, (b) note contradictions, (c) highlight knowledge gaps, and (d) directly address the research query. The resulting `aggregated_text` is the primary input to both the quality validator and the report generator.

4.6 Node 6 — Quality Validation (`quality_validator`)

The validator scores the synthesis on three dimensions:

$$\text{score} = \text{Coverage}_{[0,0.4]} + \text{Evidence}_{[0,0.3]} + \text{Coherence}_{[0,0.3]} \tag{1}$$

The LLM returns a JSON object containing `score`, `feedback`, and `missing_aspects`. The conditional edge `should_retry` then routes the graph:

$$\text{next} = \begin{cases} \text{increment_retry} \rightarrow \text{web_searcher} & \text{if score} < 0.6 \text{ and retry} < \text{max_retries} \\ \text{report_generator} & \text{otherwise} \end{cases}$$

This iterative loop allows ARIA to autonomously improve its research coverage before committing to a final report.

4.7 Node 7 — Report Generation (`report_generator`)

The final node instructs the LLM to produce a structured JSON report with the following schema:

```
{
  "title":         str,
  "abstract":      str,         # 2-3 sentence overview
  "key_findings": [str, ...], # 5-8 evidence-based bullet points
  "conclusion":    str,         # 1 well-developed paragraph
  "sources":       [{"title": str, "url": str}, ...]
}
```

Listing 2: Report JSON schema

The report is rendered to Markdown and stored in `state["report_markdown"]` for display in the UI and PDF export. A robust fallback constructs the report from sentence splits of the aggregated text if JSON parsing fails.

5 Extension — Follow-up Q&A

ARIA implements the **Follow-up Q&A** extension. After the research report is generated, the user can ask any number of follow-up questions through a chat interface

(`st.chat_input`). Each question is answered by `qa_agent.py`, which grounds the LLM strictly in the report Markdown and the full aggregated synthesis. The system prompt explicitly instructs the model to respond with a fixed refusal message if the answer is not present in the provided content, preventing hallucination beyond the researched sources.

The full Q&A history is saved to `state["qa_history"]` and is included in the PDF export.

6 Implementation Details

6.1 Technology Stack

Table 2: Technology stack for Milestone 2

Component	Library / Tool	Purpose
Agentic Framework	LangGraph $\geq$ 0.2.0	Graph orchestration, state management
LLM Interface	LangChain OpenAI $\geq$ 0.2.0	ChatOpenAI pointing at Groq base URL
LLM	Groq — Llama 3.3 70B Versatile	Free-tier LLM inference
Web Search	`ddgs` $\geq$ 9.0.0	DuckDuckGo search (no API key)
Wikipedia	`wikipedia` package	High-trust encyclopaedic source
HTML Parsing	BeautifulSoup4 $\geq$ 4.12	Full page text extraction
HTTP Client	urllib3	Robust URL fetching with timeout
PDF Export	fpdf2 $\geq$ 2.7.0	Report + Q&A history PDF generation
Frontend UI	Streamlit	Multi-page web application
Hosting	Streamlit Community Cloud	Public deployment

6.2 LLM Integration

`src/agent/llm.py` wraps Groq's OpenAI-compatible endpoint using `ChatOpenAI` from LangChain:

```python
from langchain_openai import ChatOpenAI

GROQ_BASE_URL = "https://api.groq.com/openai/v1"
GROQ_MODEL    = "llama-3.3-70b-versatile"

def get_llm(temperature: float = 0.0, api_key: str = None) -> ChatOpenAI:
    return ChatOpenAI(
        model=GROQ_MODEL,
        api_key=api_key,
        base_url=GROQ_BASE_URL,
        temperature=temperature,
        max_tokens=4096,
```

```
        max_retries=3,
        request_timeout=60,
    )
```

Listing 3: Groq LLM client (llm.py)

All nodes call `_get_llm(state)` which retrieves the API key from `state["api_key"]`, ensuring the key flows through the graph without being hard-coded or stored globally.

6.3 Robustness and Error Handling

Every node wraps its primary logic in a `try/except` block. Errors are appended to `state["errors"]` (an `Annotated[List[str], operator.add]` field that accumulates across nodes) rather than raising exceptions that would halt the graph. The UI renders an error panel via `render_error_panel` if errors are present, allowing the user to inspect issues without losing the research output.

Specific robustness measures include:

- DuckDuckGo search uses a two-backend retry (`html` → `api`) with exponential back-off.
- Failed URLs are tracked in `failed_urls` and skipped on retries.
- JSON extraction uses a fallback brace-scanning parser if `json.loads` fails on the raw LLM response.
- The quality validator defaults to a safe score of 0.7 if its own LLM call fails, preventing an infinite retry loop.
- A 1-second sleep between Groq API calls respects free-tier rate limits.

6.4 Source Quality Scoring

`web_search.py` assigns each retrieved URL a heuristic quality score:

$$Q(u) = 0.4 + \mathbb{1}[\text{HTTPS}] \cdot 0.05 + \begin{cases} 0.30 & \text{high-trust domain} \\ 0.15 & \text{medium-trust TLD} \\ 0 & \text{otherwise} \end{cases} + \mathbb{1}[\text{depth} \leq 3] \cdot 0.10 \quad (2)$$

High-trust domains include `wikipedia.org`, `arxiv.org`, `pubmed.ncbi.nlm.nih.gov`, major university domains (`.edu`, `.ac.uk`), and leading publishers.

6.5 Deployment

The application is deployed on Streamlit Community Cloud. The Groq API key is stored securely in `.streamlit/secrets.toml` (gitignored). Users may alternatively provide their key at runtime via a password input in the sidebar.

Live URL: github.com/oksaumya/ARIA

7 Input–Output Specification

Table 3: Input–output specification for Milestone 2

Input	Open-ended natural language research query (any topic)
Outputs	**Structured report:** Title, Abstract, Key Findings (5–8 bullets), Conclusion, Sources with URLs **Quality metrics:** Coverage score, source count, iteration count **Source details:** Title, URL, and snippet for each retrieved source **Follow-up Q&A:** Chat answers grounded in the report **PDF export:** Full report + Q&A history as downloadable PDF **Session history:** Last 10 research sessions accessible from the sidebar

8 Results and Evaluation

8.1 Functional Output Description

For any research query, ARIA produces:

1. A **structured research report** in Markdown, rendered directly in the Streamlit interface.
2. **Quality metadata**: coverage score (as a percentage), number of sources retrieved, and the number of research iterations performed.
3. An expandable **source details** panel listing each source with its URL and search snippet.
4. A **downloadable PDF** of the report and Q&A history.

8.2 Sample Test Cases

Table 4: Representative test queries and observed behaviour

Query	Observed Output Quality	Score	Iterations
"Latest advances in CRISPR gene editing"	5 sub-queries generated; 12 sources retrieved from arXiv, Wikipedia, and Nature; coherent 5-paragraph synthesis; 8 key findings; no retry required.	82%	1
"Quantum computing applications in finance"	Initial score 55%; one retry added 4 new DuckDuckGo sources; final score 74%; report includes concrete algorithmic examples.	74%	2
"History of the Roman Empire"	10+ Wikipedia sources; high coherence; well-structured timeline in key findings.	88%	1
"Obscure niche topic (low web coverage)"	Retry triggered; max retries reached; report generated from available sources with caveat in conclusion.	48%	3

8.3 Limitations

Rate limits on free tier

Groq's free tier imposes token-per-minute limits. Heavy queries with many sources

may trigger 429 errors, handled by LangChain's built-in retry with back-off.

DuckDuckGo scraping fragility
DuckDuckGo does not offer a stable free API; the `ddgs` library scrapes HTML responses. Structural changes to the DuckDuckGo website may break results.

Content behind paywalls
Many academic sources (e.g., IEEE, Elsevier) are behind paywalls; the content fetcher retrieves only the abstract or a teaser, reducing depth for highly technical queries.

Context window limits
Aggregating many long sources may exceed the model's context window. The 3,000-character truncation per source mitigates this but may lose detail.

Language support
The pipeline is optimised for English. Non-English sources or queries may produce degraded output.

Session persistence
Research history uses `st.session_state` and is therefore lost on browser refresh. True persistence would require a database backend.

9 Comparison: Milestone 1 vs Milestone 2

Table 5: Feature comparison between Milestone 1 (NLP baseline) and Milestone 2 (ARIA agent)

Dimension	Milestone 1	Milestone 2 (ARIA)
Input	Uploaded document or Wikipedia topic	Open-ended natural language query
Information source	User-provided / Wikipedia only	Live web (DuckDuckGo + Wikipedia)
Summarisation	Extractive (TextRank)	Abstractive (Llama 3.3 70B)
Topic discovery	Unsupervised LDA	LLM-driven query decomposition
Multi-source reasoning	None	Synthesis across up to 15 sources
Quality control	Manual inspection	Automated scoring + iterative retry
Output format	Keywords, topic clusters, extractive summary	Structured report (Title, Abstract, Findings, Conclusion, Sources)
Follow-up interaction	None	Grounded Q&A chat
PDF export	None	Full report + Q&A history
LLM usage	None (classical NLP only)	Llama 3.3 70B via Groq (free tier)
Agentic framework	None	LangGraph stateful graph

10 Conclusion

This report presented ARIA (Autonomous Research Intelligence Assistant), the Milestone 2 extension of the Intelligent Research Topic Analysis project. The system demonstrates a clear and fully functional progression from classical NLP (Milestone 1) to autonomous agentic AI (Milestone 2).

ARIA's LangGraph-orchestrated pipeline — spanning research planning, multi-source web retrieval, full-page content fetching, per-source LLM summarisation, quality-gated iterative refinement, and structured report generation — fulfils all Milestone 2 functional and technical requirements. The grounded follow-up Q&A extension further extends the system beyond the baseline deliverables. The application is publicly deployed on Streamlit Community Cloud, satisfying the mandatory hosting constraint, and operates entirely within free-tier API budgets.

The modular codebase (separating state, graph topology, node logic, LLM client, web search, and report rendering into distinct modules) ensures maintainability and provides a strong foundation for future extensions such as persistent session storage, PDF document ingestion within the agentic loop, or integration with additional free-tier search APIs.